UNIT-I

INTRODUCTION TO DATABASE SYSTEM

What is Data?

In simple words, data can be facts related to any object in consideration. For example, your name, age, height, weight, etc. are some data related to you. A picture, image, file, pdf, etc. can also be considered data.

What is Database?

A database is a collection of data, typically describing the activities of one or more related organizations. For example, a university database might contain information about the following:

Entities such as students, faculty, courses, and classrooms.

Relationships between entities, such as student's enrolment in courses, faculty teaching courses, and the use of rooms for courses.

Database Management System (DBMS):

A Database Management System (DBMS) is a software system that is designed to manage and organize data in a structured manner. It allows users to create, modify, and query a database, as well as manage the security and access controls for that database.

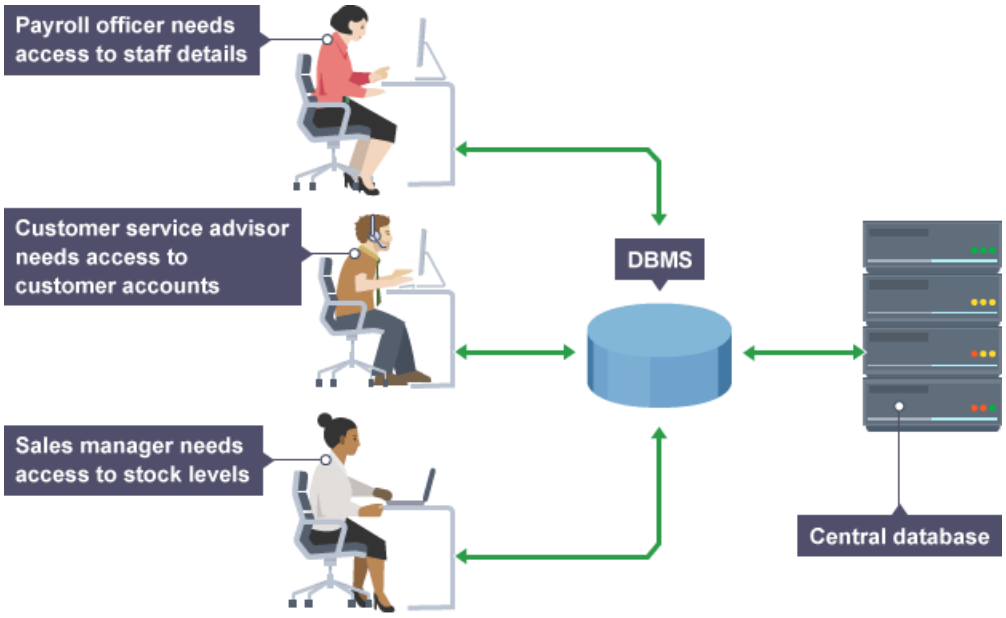


Figure : Employees are accessing Data through DBMS

For example, within a company there are different departments, as well as customers, who each need to see different kinds of data. Each employee in the company will have different levels of access to the database with their own customized front-end application. In a database, data is organized strictly in row

and column format. The rows are called Tuple or Record. The data items within one row may belong to different data types. On the other hand, the columns are often called Domain or Attribute. All the data items within a single attribute are of the same data type.

Applications of DBMS:

There are different fields where a database management system is utilized. Following are a few applications which utilize the information base administration framework –

- **Railway Reservation System**: In the rail route reservation framework, the information base is needed to store the record or information of ticket appointments, status about train's appearance, and flight. Additionally, if trains get late, individuals become acquainted with it through the information base update.
- **Library Management System:** There are lots of books in the library so; it is difficult to store the record of the relative multitude of books in a register or duplicate. Along these lines, the data set administration framework (DBMS) is utilized to keep up all the data identified with the name of the book, issue date, accessibility of the book, and its writer.
- **Banking:** Database the executive's framework is utilized to store the exchange data of the client in the information base.
- **Education Sector:** Presently, assessments are led online by numerous schools and colleges. They deal with all assessment information through the data set administration framework (DBMS). In spite of that understudy's enlistments subtleties, grades, courses, expense, participation, results, and so forth all the data is put away in the information base.
- **Credit card exchanges :** The database Management framework is utilized for buying on charge cards and age of month-to-month proclamations.
- **Social Media Sites:** We all utilization of online media sites to associate with companions and to impart our perspectives to the world. Every day, many people group pursue these online media accounts like Pinterest, Facebook, Twitter, and Google in addition to. By the utilization of the data set administration framework, all the data of clients are put away in the information base and, we become ready to interface with others.
- **Broadcast communications:** Without DBMS any media transmission organization can't think. The Database the executive's framework is fundamental for these organizations to store the call subtleties and month to month postpaid bills in the information base.
- **Account:** The information base administration framework is utilized for putting away data about deals, holding and acquisition of monetary instruments, for example, stocks and bonds in a data set.
- **Online Shopping:** These days, web-based shopping has become a major pattern. Nobody needs to visit the shop and burn through their time. Everybody needs to shop through web-based shopping sites, (for example, Amazon, Flipkart, Snapdeal) from home. So all the items are sold and added uniquely with the assistance of the information base administration framework (DBMS). Receipt charges, instalments, buy data these are finished with the assistance of DBMS.
- **Human Resource Management:** Big firms or organizations have numerous specialists or representatives working under them. They store data about worker's compensation, assessment, and work with the assistance of an information base administration framework (DBMS).

- **Manufacturing**: Manufacturing organizations make various kinds of items and deal them consistently. To keep the data about their items like bills, acquisition of the item, amount, inventory network the executives, information base administration framework (DBMS) is utilized.
- **Airline Reservation System –** This framework is equivalent to the railroad reservation framework. This framework additionally utilizes an information base administration framework to store the records of flight take off, appearance, and defer status.
- **Healthcare:** DBMS is used in healthcare to manage patient data, medical records, and billing information.
- **Data retrieval:** DBMS provides a way to retrieve data quickly and easily using search queries.
- **Data manipulation:** DBMS provides tools to manipulate data, such as sorting, filtering, and aggregating data.
- **Security:** DBMS provides security features to ensure that only authorized users have access to the data.
- **Data backup and recovery:** DBMS provides tools to back up data and recover it in case of system failures or data loss.
- **Multi-user access**: DBMS allows multiple users to access and modify data simultaneously.
- **Reporting and analysis:** DBMS provides tools to generate reports and analyze data to gain insights and make informed decisions.

Purpose of Database Systems:

This typical file-processing system is supported by a conventional operating system. The system stores permanent records in various files, and it needs different application programs to extract records from, and add records to, the appropriate files. Before database management systems (DBMSs) were introduced, organizations usually stored information in such systems. Keeping organizational information in a file processing system has a number of major disadvantages:

Data redundancy and inconsistency: Since different programmers create the files and application programs over a long period, the various files are likely to have different structures and the programs may be written in several programming languages. Moreover, the same information may be duplicated in several places (files).

For example, if a student has a double major (say, music and mathematics) the address and telephone number of that student may appear in a file that consists of student records of students in the Music department and in a file that consists of student records of students in the mathematics department. This redundancy leads to higher storage and access cost. In addition, it may lead to data inconsistency; that is, the various copies of the same data may no longer agree. For example, a changed student address may be reflected in the Music department records but not elsewhere in the system.

Difficulty in accessing data: Suppose that one of the university clerks needs to find out the names of all students who live within a particular postal-code area. The clerk asks the data-processing department to generate such a list. Because the designers of the original system did not anticipate this request, there is no application program on hand to meet it. There is, however, an application program to generate the list of *all* students. The university clerk has now two choices: either obtain the list of all students and extract the needed information manually or ask a programmer to write the necessary application program. Both alternatives are obviously unsatisfactory. Suppose that such a program is written, and that, several days later, the same clerk needs to trim that list to include only those students who have taken at least 60 credit hours. As expected, a program to generate such a list does not exist. Again, the clerk has the

preceding two options, neither of which is satisfactory. The point here is that conventional file-processing environments do not allow needed data to be retrieved in a convenient and efficient manner. More responsive data-retrieval systems are required for general use.

Data isolation: Because data are scattered in various files, and files may be in different formats, writing new application programs to retrieve the appropriate data is difficult.

Integrity problems: The data values stored in the database must satisfy certain types of consistency constraints. Suppose the university maintains an account for each department, and records the balance amount in each account. Suppose also that the university requires that the account balance of a department may never fall below zero. Developers enforce these constraints in the system by adding appropriate code in the various application programs. However, when new constraints are added, it is difficult to change the programs to enforce them. The problem is compounded when constraints involve several data items from different files.

Atomicity problems: A computer system, like any other device, is subject to failure. In many applications, it is crucial that, if a failure occurs, the data be restored to the consistent state that existed prior to the failure. Consider a program to transfer $500 from the account balance of department *A* to the account balance of department *B*. If a system failure occurs during the execution of the program, it is possible that the $500 was removed from the balance of department *A* but was not credited to the balance of department *B*, resulting in an inconsistent database state. Clearly, it is essential to database consistency that either both the credit and debit occur, or that neither occur. That is, the funds transfer must be *atomic*—it must happen in its entirety or not at all. It is difficult to ensure atomicity in a conventional file-processing system.

Concurrent-access anomalies: For the sake of overall performance of the system and faster response, many systems allow multiple users to update the data simultaneously. Indeed, today, the largest Internet retailers may have millions of accesses per day to their data by shoppers. In such an environment, interaction of concurrent updates is possible and may result in inconsistent data. Consider department *A*, with an account balance of $10,000. If two department clerks debit the account balance (by say $500 and $100, respectively) of department *A* at almost exactly the same time, the result of the concurrent executions may leave the budget in an incorrect (or inconsistent) state. Suppose that the programs executing on behalf of each withdrawal read the old balance, reduce that value by the amount being withdrawn, and write the result back. If the two programs run concurrently, they may both read the value $10,000, and write back $9500 and $9900, respectively. Depending on which one writes the value last, the account balance of department *A* may contain either $9500 or $9900, rather than the correct value of $9400. To guard against this possibility, the system must maintain some form of supervision. But supervision is difficult to provide because data may be accessed by many different application programs that have not been coordinated previously. As another example, suppose a registration program maintains a count of students registered for a course, in order to enforce limits on the number of students registered. When a student registers, the program reads the current count for the courses, verifies that the count is not already at the limit, adds one to the count, and stores the count back in the database. Suppose two students register concurrently, with the count at (say) 39. The two program executions may both read the value 39, and both would then write back 40, leading to an incorrect increase of only 1, even though two students successfully registered for the course and the count should be 41.

Furthermore, suppose the course registration limit was 40; in the above case both students would be able to register, leading to a violation of the limit of 40 students.

Security problems: Not every user of the database system should be able to access all the data. For example, in a university, payroll personnel need to see only that part of the database that has financial

information. They do not need access to information about academic records. But, since application programs are added to the file-processing system in an ad hoc manner, enforcing such security constraints is difficult. These difficulties, among others, prompted the development of database systems. In what follows, we shall see the concepts and algorithms that enable database systems to solve the problems with file-processing systems.

Advantages of DBMS:

- **Controlling of Redundancy:** Data redundancy refers to the duplication of data (i.e storing same data multiple times). In a database system, by having a centralized database and centralized control of data by the DBA the unnecessary duplication of data is avoided. It also eliminates the extra time for processing the large volume of data. It results in saving the storage space.
- **Improved Data Sharing:** DBMS allows a user to share the data in any number of application programs.
- **Data Integrity:** Integrity means that the data in the database is accurate. Centralized control of the data helps in permitting the administrator to define integrity constraints to the data in the database. For example: in customer database we can enforce an integrity that it must accept the customer only from Noida and Meerut city.
- **Security:** Having complete authority over the operational data, enables the DBA in ensuring that the only mean of access to the database is through proper channels. The DBA can define authorization checks to be carried out whenever access to sensitive data is attempted.
- **Data Consistency:** By eliminating data redundancy, we greatly reduce the opportunities for inconsistency. For example: is a customer address is stored only once, we cannot have disagreement on the stored values. Also updating data values is greatly simplified when each value is stored in one place only. Finally, we avoid the wasted storage that results from redundant data storage.
- **Efficient Data Access:** In a database system, the data is managed by the DBMS and all access to the data is through the DBMS providing a key to effective data processing
- **Enforcements of Standards:** With the centralized of data, DBA can establish and enforce the data standards which may include the naming conventions, data quality standards etc.
- **Data Independence:** In a database system, the database management system provides the interface between the application programs and the data. When changes are made to the data representation, the meta data obtained by the DBMS is changed but the DBMS is continuing to provide the data to application program in the previously used way. The DBMs handles the task of transformation of data wherever necessary.
- **Reduced Application Development and Maintenance Time:** DBMS supports many important functions that are common to many applications, accessing data stored in the DBMS, which facilitates the quick development of application.

File System vs DBMS

File System	DBMS
A file system is a programme that manages and organises files on a storage medium. It controls data retrieval and archiving.	A database management system, also known as DBMS, is a type of software. It can be used for constructing, accessing, and managing databases
Data presentation and storage information are provided by the file system	DBMS provides an abstract representation of the data, hiding the specifics.
It is impossible to store and retrieve data efficiently in a file system	Because there are numerous ways to store and retrieve data, DBMS is efficient to utilise.
Because there are numerous ways to store and retrieve data, DBMS is efficient to utilise.	It doesn’t provide methods for data recovery
It doesn’t provide methods for data recover	Data in DBMS can be recovered from backups.
The file system lacks a crash recovery mechanism.	A crash recovery method is offered by DBMS.
It can be difficult to maintain file system security.	DBMS has good security measures.
In a file management system, data redundancy is higher.	The DBMS system’s data redundancy is poor.
The file system has a higher rate of data inconsistency	A database management system has a poor level of data consistency.
The security level of the file system is lower.	Highly secure database management systems are available.
You can save the data as separate data files and entities using the file system.	Database management systems hold data along with restrictions and relationships that have been established.
It is difficult to centralise in a file management system	The DBMS system makes it simple to accomplish centralization.
It doesn’t provide data backup or recovery if it is lost.	Even if data is lost, DBMS systems offer backup and recovery services.
Inefficient query processing exists in the file system.	Using the SQL language, you may query data in a database.
This system does not support concurrency.	DBMS systems offer a concurrency facility.

View of Data:

A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data. A major purpose of a database system is to provide users with an *abstract* view of the data. That is, the system hides certain details of how the data are stored and maintained.

Data Abstraction:

For the system to be usable, it must retrieve data efficiently. The need for efficiency has led designers to use complex data structures to represent data in the database. Since many database-system users are not computer trained, developers hide the complexity from users through several levels of abstraction, to simplify users' interactions with the system:

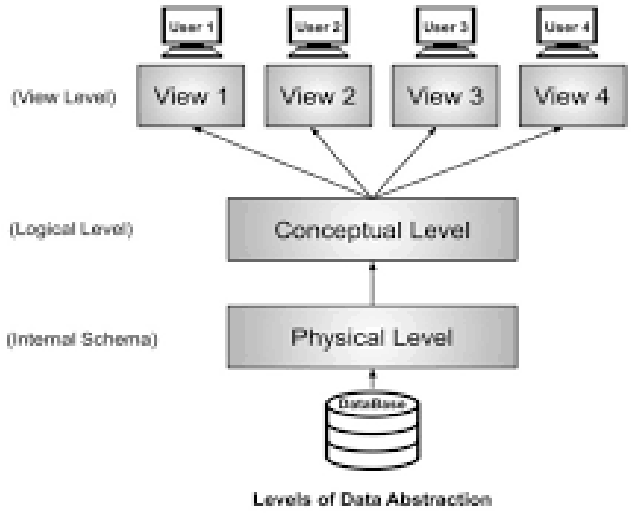


Figure: Levels of Abstraction in a DBMS

• **Physical level (or Internal View / Schema):** The lowest level of abstraction describes *how* the data are actually stored. The physical level describes complex low-level data structures in detail.

• **Logical level (or Conceptual View / Schema):** The next-higher level of abstraction describes *what* data are stored in the database, and what relationships exist among those data. The logical level thus describes the entire database in terms of a small number of relatively simple structures. Although implementation of the simple structures at the logical level may involve complex physical-level structures, the user of the logical level does not need to be aware of this complexity. This is referred to as physical data independence. Database administrators, who must decide what information to keep in the database, use the logical level of abstraction.

• **View level (or External View / Schema):** The highest level of abstraction describes only part of the entire database. Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database. Many users of the database system do not need all this information; instead, they need to access only a part of the database. The view level of abstraction exists to simplify their interaction with the system. The system may provide many views for the same database. Figure shows the relationship among the three levels of abstraction.

An analogy to the concept of data types in programming languages may clarify the distinction among levels of abstraction. Many high-level programming languages support the notion of a structured type. For example, we may describe a record as follows:

type *instructor* = record
ID : char (5);
name : char (20);
dept name : char (20);
salary : numeric (8,2);
end;

This code defines a new record type called *instructor* with four fields. Each field has a name and a type associated with it. A university organization may have several such record types, including Database DISK

- *department*, with fields *dept_name*, *building*, and *budget*
- *course*, with fields *course_id*, *title*, *dept_name*, and *credits*
- *student*, with fields *ID*, *name*, *dept_name*, and *tot_cred*

At the physical level, an *instructor*, *department*, or *student* record can be described as a block of consecutive storage locations. The compiler hides this level of detail from programmers. Similarly, the database system hides many of the lowest-level storage details from database programmers. Database administrators, on the other hand, may be aware of certain details of the physical organization of the data. At the logical level, each such record is described by a type definition, as in the previous code segment, and the interrelationship of these record types is defined as well. Programmers using a programming language work at this level of abstraction. Similarly, database administrators usually work at this level of abstraction.

Finally, at the view level, computer users see a set of application programs that hide details of the data types.

At the view level, several views of the database are defined, and a database user sees some or all of these views. In addition to hiding details of the logical level of the database, the views also provide a security mechanism to prevent users from accessing certain parts of the database. For example, clerks in the university registrar office can see only that part of the database that has information about students; they cannot access information about salaries of instructors.

Instances and Schemas:

Databases change over time as information is inserted and deleted. The collection of information stored in the database at a particular moment is called an instance of the database. The overall design of the database is called the database schema. Schemas are changed infrequently, if at all. The concept of database schemas and instances can be understood by analogy to a program written in a programming language. A database schema corresponds to the variable declarations (along with associated type definitions) in a program. Each variable has a particular value at a given instant. The values of the variables in a program at a point in time correspond to an *instance* of a database schema. Database systems have several schemas, partitioned according to the levels of abstraction. The physical schema describes the database design at the physical level, while the logical schema describes the database design at the logical level. A database may also have several schemas at the view level, sometimes called subschemas, which describe different views of the database. Of these, the logical schema is by far the most important, in terms of its effect on application programs, since programmers construct applications by using the logical schema. The physical schema is hidden beneath the logical schema, and can usually be changed easily without affecting application programs. Application programs are said to exhibit physical data independence if they do not depend on the physical schema, and thus need not be rewritten if the physical schema changes.

Example for Instance:

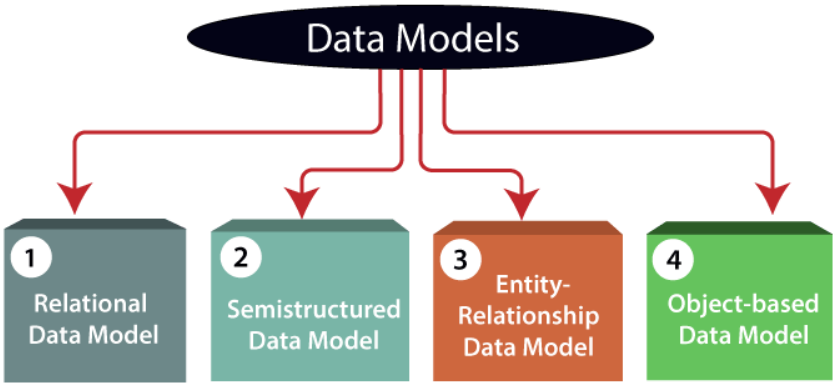
Let's see the table with the data now. At this moment the table contains two rows (records). This is the the current instance of the table "employee" because this is the data that is stored in this table at this particular moment of time.

EMP_NAME	EMP_ID	EMP_ADDRESS	EMP_CONTACT
Chaitanya	101	Noida	95********
Ajeet	102	Delhi	99********

Let's take another example: Let's say we have a single table student in the database, today the table has 100 records, so today the instance of the database has 100 records. We are going to add another 100 records in this table by tomorrow so the instance of database tomorrow will have 200 records in table. In short, at a particular moment the data stored in database is called the instance, this changes over time as and when we add, delete or update data in the database.

Data models:

Data Model is the modeling of the data description, data semantics, and consistency constraints of the data. It provides the conceptual tools for describing the design of a database at each level of data abstraction. Therefore, there are following four data models used for understanding the structure of the database:



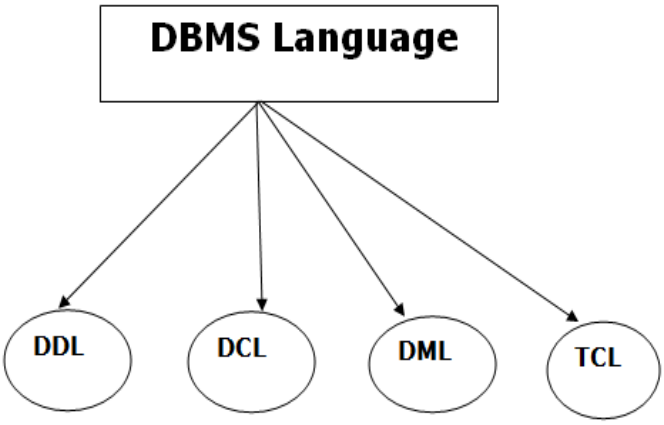
1. **Relational Data Model:** This type of model designs the data in the form of rows and columns within a table. Thus, a relational model uses tables for representing data and in-between relationships. Tables are also called relations. This model was initially described by Edgar F. Codd, in 1969. The relational data model is the widely used model which is primarily used by commercial data processing applications.
2. **Semistructured Data Model:** This type of data model is different from the other three data models (explained above). The semistructured data model allows the data specifications at places where the individual data items of the same type may have different attributes sets. The Extensible Markup Language, also known as XML, is widely used for representing the semistructured data. Although XML was initially designed for including the markup information to the text document, it gains importance because of its application in the exchange of data.
3. **Entity-Relationship Data Model:** An ER model is the logical representation of data as objects and relationships among them. These objects are known as entities, and relationship is an association

among these entities. This model was designed by Peter Chen and published in 1976 papers. It was widely used in database designing. A set of attributes describe the entities. For example, student_name, student_id describes the 'student' entity. A set of the same type of entities is known as an 'Entity set', and the set of the same type of relationships is known as 'relationship set'

4. **Object-based Data Model:** An extension of the ER model with notions of functions, encapsulation, and object identity, as well. This model supports a rich type system that includes structured and collection types. Thus, in 1980s, various database systems following the object-oriented approach were developed. Here, the objects are nothing but the data carrying its properties.

DBMS languages :

Database languages are used to read, update and store data in a database. There are several such languages that can be used for this purpose; one of them is SQL (Structured Query Language).



1. DDL – Data Definition Language: (CREATE,DROP,ALTER,TRUNCATE,COMMENT,RENAME)
2. DML – Data Manipulation Language: (INSERT, UPDATE,DELETE)
3. DCL – Data Control Language: (GRANT,REVOKE)
4. TCL-Transaction Control Language: (COMMIT,ROLLBACK)

1. DDL(Data Definition Language) :

DDL or Data Definition Language actually consists of the SQL commands that can be used to define the database schema. It simply deals with descriptions of the database schema and is used to create and modify the structure of database objects in the database.

CREATE – it is used to create the database or its objects (like table, index, function, views, store procedure and triggers).

There are two CREATE statements available in SQL:

- CREATE DATABASE
- CREATE TABLE

CREATE DATABASE:

A Database is defined as a structured set of data. So, in SQL the very first step to store the data in a well structured manner is to create a database. The CREATE DATABASE statement is used to create a new database in SQL.

Syntax:

CREATE DATABASE database_name;

Example:

SQL> CREATE DATABASE Employee;

In order to get the list of all the databases, you can use SHOW DATABASES statement.

Example:

SQL> SHOW DATABASES;

CREATE TABLE:

The CREATE TABLE statement is used to create a table in SQL. We know that a table comprises of rows and columns. So while creating tables we have to provide all the information to SQL about the names of the columns, type of data to be stored in columns, size of the data etc. Let us now dive into details on how to use CREATE TABLE statement to create tables in SQL.

Syntax:

```
CREATE TABLE table_name
(
column1 data_type(size),
column2 data_type(size),
column3 data_type(size),
....
);
```

Example: This query will create a table named Students with three columns, ROLL_NO, NAME and SUBJECT.

```
CREATE TABLE Students
(
ROLL_NO int(3),
NAME varchar(20),
SUBJECT varchar(20),
);
```

DROP:

DROP is used to delete a whole database or just a table.The DROP statement destroys the objects like an existing database, table, index, or view

A DROP statement in SQL removes a component from a relational database management system (RDBMS).

Syntax:

DROP object object_name;

Example:

DROP TABLE table_name;

table_name: Name of the table to be deleted.

DROP DATABASE database_name;

database_name: Name of the database to be deleted.

TRUNCATE

It is used to remove all records from a table, including all spaces .The TRUNCATE TABLE mytable statement is logically (though not physically) equivalent to the DELETE FROM mytable statement (without a WHERE clause).

Syntax:

TRUNCATE TABLE table_name;

DROP vs TRUNCATE

- Truncate is normally ultra-fast and its ideal for deleting data from a temporary table.
- Truncate preserves the structure of the table for future use, unlike drop table where the table is deleted with its full structure.
- Table or Database deletion using DROP statement cannot be rolled back, so it must be used wisely.

To delete the whole database:

DROP DATABASE student_data;

After running the above query whole database will be deleted.

To truncate Student_details table from student_data database.

TRUNCATE TABLE Student_details;

ALTER (ADD, DROP, MODIFY):

ALTER TABLE is used to add, delete/drop or modify columns in the existing table. It is also used to add and drop various constraints on the existing table.

ALTER TABLE – ADD:

ADD is used to add columns into the existing table. Sometimes we may require to add additional information, in that case we do not require to create the whole database again, ADD comes to our rescue.

Syntax:

ALTER TABLE table_name
ADD (Columnname_1 datatype,
Columnname_2 datatype,
…
Columnname_n datatype);

ALTER TABLE – DROP :

DROP COLUMN is used to drop column in a table. Deleting the unwanted columns from the table.

Syntax:

ALTER TABLE table_name
DROP COLUMN column_name;

ALTER TABLE-MODIFY

It is used to modify the existing columns in a table. Multiple columns can also be modified at once.

ALTER TABLE table_name
MODIFY column_name column_type;

Comments:

As is any programming languages comments matter a lot in SQL also. In this set we will learn about writing comments in any SQL snippet.

Comments can be written in the following three formats:

- Single line comments.
- Multi line comments
- In line comments

2. DML(Data Manipulation Language) :

The SQL commands that deals with the manipulation of data present in the database belong to DML or Data Manipulation Language and this includes most of the SQL statements.

SELECT Statement:

select statement is used to fetch data from relational database. A relational database is organized collection of data. As we know that data is stored inside tables in a database. SQL select statement or SQL select query is used to fetch data from one or more than one tables.

Syntax:

One column: Here column_name is the name of the column for which we need to fetch data and table_name is the name of the table in the database.

SELECT column_name FROM table_name;

More than one columns:

SELECT column_name_1, column_name_2, ... FROM table_name;

To fetch the entire table or all the fields in the table:

SELECT * FROM table_name;

Example:

SELECT EMP_NAME FROM EMPLOYEES;

To fetch the entire EMPLOYEES table:

SELECT * FROM EMPLOYEES;

Query to fetch the fields ROLL_NO, NAME, AGE from the table Student:

SELECT ROLL_NO, NAME, AGE FROM Student;

INSERT INTO Statement

The INSERT INTO statement of SQL is used to insert a new row in a table. There are two ways of using INSERT INTO statement for inserting rows:

- **Only values**: First method is to specify only the value of data to be inserted without the column names.
  INSERT INTO table_name VALUES (value1, value2, value3,…);

- **Column names and values both:** In the second method we will specify both the columns which we want to fill and their corresponding values as shown below:
  INSERT INTO table_name (column1, column2, column3,..) VALUES (value1, value2, value3,..);

Example:
Method 1 (Inserting only values) :
INSERT INTO Student VALUES ('5','HARSH','WEST BENGAL','XXXXXXXXXX','19');
Method 2 (Inserting values in only specified columns):
INSERT INTO Student (ROLL_NO, NAME, Age) VALUES ('5','PRATIK','19');

UPDATE Statement
The UPDATE statement in SQL is used to update the data of an existing table in database. We can update single columns as well as multiple columns using UPDATE statement as per our requirement.
Syntax:
UPDATE TableName
SET column_name1 = value, column_name2 = value....
WHERE condition;

Example1:
SQL> UPDATE EMPLOYEES
SET EMP_SALARY = 10000
WHERE EMP_AGE > 25;

Example2:
SQL> UPDATE EMPLOYEES
SET EMP_SALARY = 120000
WHERE EMP_NAME = 'Apoorv';

DELETE Statement
The DELETE Statement in SQL is used to delete existing records from a table. We can delete a single record or multiple records depending on the condition we specify in the WHERE clause.
Syntax:
DELETE FROM table_name WHERE some_condition;

Deleting single record: Delete the rows where NAME = 'Ram'. This will delete only the first row.
DELETE FROM Student WHERE NAME = 'Ram';

Deleting multiple records: Delete the rows from the table Student where Age is 20. This will delete 2 **rows(third row and fifth row).**
DELETE FROM Student WHERE Age = 20;

Delete all of the records: There are two queries to do this as shown below,
query1: "DELETE FROM Student";
query2: "DELETE * FROM Student";

3.DCL(Data Control Language) :

DCL includes commands such as GRANT and REVOKE which mainly deals with the rights, permissions and other controls of the database system.

Examples of DCL commands:

GRANT-gives user's access privileges to database.

REVOKE-withdraw user's access privileges given by using the GRANT command.

4.TCL(transaction Control Language) :

TCL commands deals with the transaction within the database.

Examples of TCL commands:

COMMIT– commits a Transaction.

ROLLBACK– rollbacks a transaction in case of any error occurs.

SAVEPOINT–sets a savepoint within a transaction.

SET TRANSACTION–specify characteristics for the transaction.

Structure of DBMS:

The database system is divided into three components: Query Processor, Storage Manager, and Disk Storage. These are explained as following below.

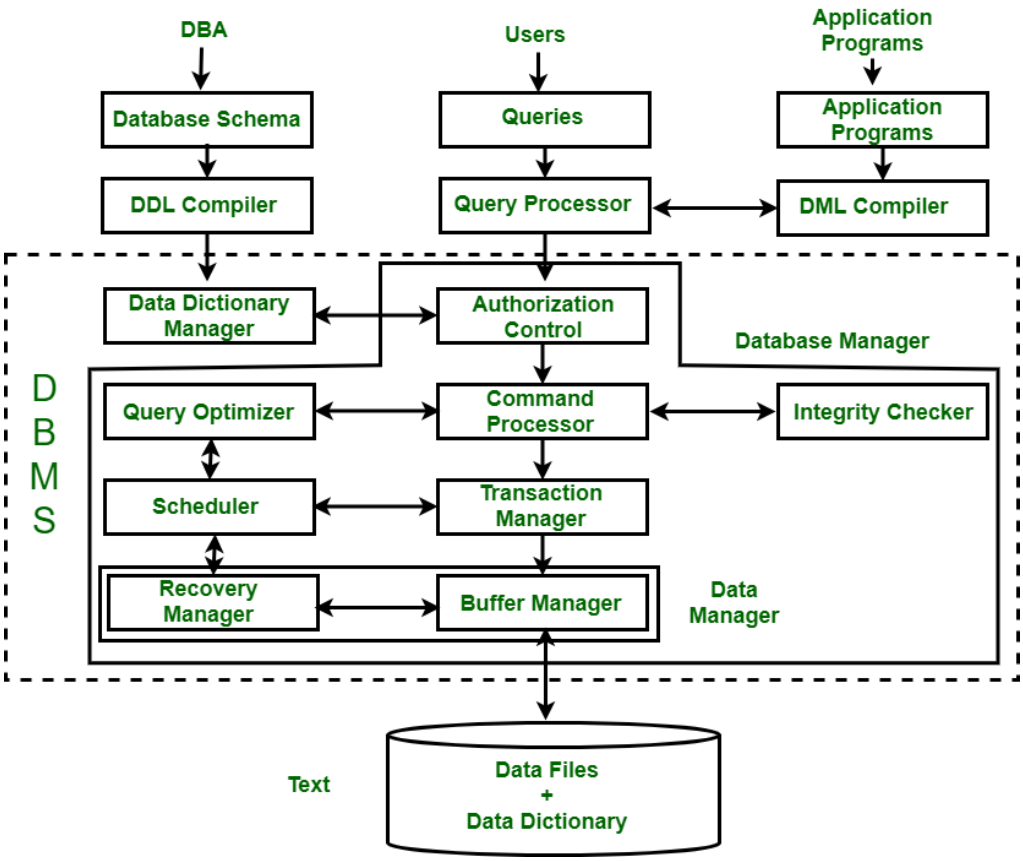


Figure: Structure of DBMS

1. Query Processor: It interprets the requests (queries) received from end user via an application program into instructions. It also executes the user request which is received from the DML compiler. Query Processor contains the following components –

- DML Compiler: It processes the DML statements into low level instruction (machine language), so that they can be executed.
- DDL Interpreter: It processes the DDL statements into a set of table containing meta data (data about data).
- Embedded DML Pre-compiler: It processes DML statements embedded in an application program into procedural calls.
- Query Optimizer: It executes the instruction generated by DML Compiler.

2. Storage Manager: Storage Manager is a program that provides an interface between the data stored in the database and the queries received. It is also known as Database Control System. It maintains the consistency and integrity of the database by applying the constraints and executing the DCL statements. It is responsible for updating, storing, deleting, and retrieving data in the database. It contains the following components –

- Authorization Manager: It ensures role-based access control, i.e,. checks whether the particular person is privileged to perform the requested operation or not.
- Integrity Manager: It checks the integrity constraints when the database is modified.
- Transaction Manager: It controls concurrent access by performing the operations in a scheduled way that it receives the transaction. Thus, it ensures that the database remains in the consistent state before and after the execution of a transaction.
- File Manager: It manages the file space and the data structure used to represent information in the database.
- Buffer Manager: It is responsible for cache memory and the transfer of data between the secondary storage and main memory.

3. Disk Storage: It contains the following components –

- Data Files: It stores the data.
- Data Dictionary: It contains the information about the structure of any database object. It is the repository of information that governs the metadata.
- Indices: It provides faster retrieval of data item.

The structure of a Database Management System (DBMS) can be divided into three main components: the Internal Level, the Conceptual Level, and the External Level.

1. Internal Level: This level represents the physical storage of data in the database. It is responsible for storing and retrieving data from the storage devices, such as hard drives or solid-state drives. It deals with low-level implementation details such as data compression, indexing, and storage allocation.
2. Conceptual Level: This level represents the logical view of the database. It deals with the overall organization of data in the database and the relationships between them. It defines the data schema, which includes tables, attributes, and their relationships. The conceptual level is independent of any specific DBMS and can be implemented using different DBMSs.
3. External Level: This level represents the user's view of the database. It deals with how users access the data in the database. It allows users to view data in a way that makes sense to them, without worrying about the underlying implementation details. The external level

provides a set of views or interfaces to the database, which are tailored to meet the needs of specific user groups.

The three levels are connected through a schema mapping process that translates data from one level to another. The schema mapping process ensures that changes made at one level are reflected in the other levels.

In addition to these three levels, a DBMS also includes a Database Administrator (DBA) component, which is responsible for managing the database system. The DBA is responsible for tasks such as database design, security management, backup and recovery, and performance tuning.

Overall, the structure of a DBMS is designed to provide a high level of abstraction to users, while still allowing low-level implementation details to be managed effectively. This allows users to focus on the logical organization of data in the database, without worrying about the physical storage or implementation details.

History of DBMS:

Data is a collection of facts and figures. The data collection was increasing day to day and they needed to be stored in a device or a software which is safer.

Charles Bachman was the first person to develop the Integrated Data Store (IDS) which was based on network data model for which he was inaugurated with the Turing Award (The most prestigious award which is equivalent to Nobel prize in the field of Computer Science.). It was developed in early 1960's.

In the late 1960's, IBM (International Business Machines Corporation) developed the Integrated Management Systems which is the standard database system used till date in many places. It was developed based on the hierarchical database model. It was during the year 1970 that the relational database model was developed by Edgar Codd. Many of the database models we use today are relational based. It was considered the standardized database model from then.
The relational model was still in use by many people in the market. Later during the same decade (1980's), IBM developed the Structured Query Language (SQL) as a part of R project. It was declared as a standard language for the queries by ISO and ANSI. The Transaction Management Systems for processing transactions was also developed by James Gray for which he was felicitated the Turing Award.

Further, there were many other models with rich features like complex queries, datatypes to insert images and many others. The Internet Age has perhaps influenced the data models much more. Data models were developed using object oriented programming features, embedding with scripting languages like Hyper Text Markup Language (HTML) for queries. With humongous data being available online, DBMS is gaining more significance day by day.

INTRODUCTION TO DATABASE DESIGN

DATABASE DESIGN:

The database design process is divided into six steps:

1.Requirement analysis:

The first step in designing a database application is to understand what data is to be stored, what users want etc.

2.Conceptual database design:

The information gathered in the first step is used to develop a high-level description of data to be stored in databases along with constraints. This step is carried out using ER model.

3.Logical database design:

DBMS is used to implement database design and covert the conceptual design to relational model. That is ER model is converted into Relational model.

4.Schema Refinement:

The relational database schema contains the different collection of relations. Schema Refinement is to analyze the problems like redundancy, inconsistency, irregularity in data and should be modified. This step uses the process of Normalization.

5.Physical Database Design:

This step of Database design is to ensure that it meets desired performance. This typically includes building indexes on tables and clustering them. This indexing makes to retrieve data more efficiently.

6.Security Design:

In any organization, there will be different user groups and roles played by different users. All the users are not allowed to access the data completely.

So, every user is granted an authority to access the data.

Conceptual Database Design - Entity Relationship(ER) Modeling:

What is ER Modeling?
A graphical technique for understanding and organizing the data independent of the actual database implementation
We need to be familiar with the following terms to go further.
Entity
Any thing that has an independent existence and about which we collect data. It is also known as entity type.
In ER modeling, notation for entity is given below.

Entity instance
Entity instance is a particular member of the entity type.
Example: A particular employee

Regular Entity

An entity which has its own key attribute is a regular entity.

Example: Employee.

Weak entity

An entity which depends on other entity for its existence and doesn't have any key attribute of its own is a weak entity.

Example : In a parent/child relationship, a parent is considered as a strong entity and the child is a weak entity.

In ER modeling, notation for weak entity is given below.

Attributes

Properties/characteristics which describe entities are called attributes.

In ER modeling, notation for attribute is given below.

Domain of Attributes

The set of possible values that an attribute can take is called the domain of the attribute. For example, the attribute day may take any value from the set {Monday, Tuesday ... Friday}. Hence this set can be termed as the domain of the attribute day.

Key attribute

The attribute (or combination of attributes) which is unique for every entity instance is called key attribute.

Example: the employee_id of an employee, pan_card_number of a person etc.If the key attribute consists of two or more attributes in combination, it is called a composite key.

In ER modeling, notation for key attribute is given below.

Simple attribute

If an attribute cannot be divided into simpler components, it is a simple attribute.

Example : employee_id of an employee.

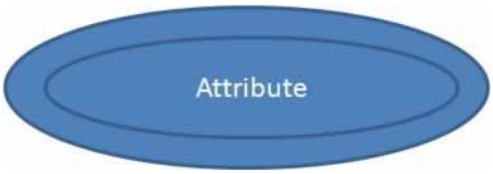
Composite attribute

If an attribute can be split into components, it is called a composite attribute.
Example: Name of the employee which can be split into First_name, Middle_name, and Last_name.

Single valued Attributes

If an attribute can take only a single value for each entity instance, it is a single valued attribute.
Example : age of a student. It can take only one value for a particular student.

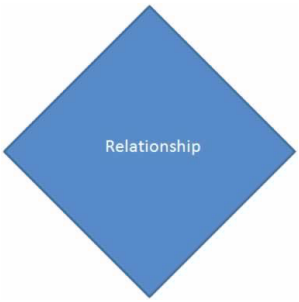
Multi-valued Attributes
If an attribute can take more than one value for each entity instance, it is a multi-valued attribute.
Example: telephone number of an employee, a particular employee may have multiple telephone numbers.
In ER modeling, notation for multi-valued attribute is given below.



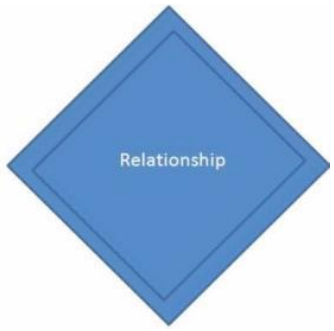
Stored Attribute
An attribute which need to be stored permanently is a stored attribute
Example: name of a student

Derived Attribute
An attribute which can be calculated or derived based on other attributes is a derived attribute.
Example: age of employee which can be calculated from date of birth and current date.
In ER modeling, notation for derived attribute is given below.

Relationships
Associations between entities are called relationships
Example : An employee works for an organization. Here "works for" is a relation between the entities employee and organization.
In ER modeling, notation for relationship is given below.



However in ER Modeling, To connect a weak Entity with others, you should use a weak relationship notation as given below



Degree of a Relationship

Degree of a relationship is the number of entity types involved. The n-ary relationship is the general form for degree n. Special cases are unary, binary, and ternary ,where the degree is 1, 2, and 3, respectively.

Example for unary relationship : An employee is a manager of another employee

Example for binary relationship : An employee works-for department.

Example for ternary relationship : customer purchase item from a shop keeper

Cardinality of a Relationship

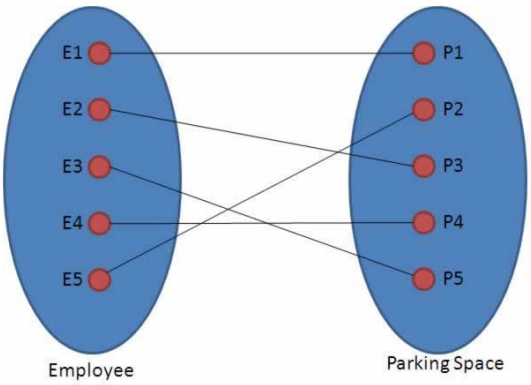
Relationship cardinalities specify how many of each entity type is allowed. Relationships can have four possible connectivities as given below.

1. One to one (1:1) relationship
2. One to many (1:N) relationship
3. Many to one (M:1) relationship
4. Many to many (M:N) relationship

The minimum and maximum values of this connectivity is called the cardinality of the relationship

Example for Cardinality – One-to-One (1:1)

Employee is assigned with a parking space.

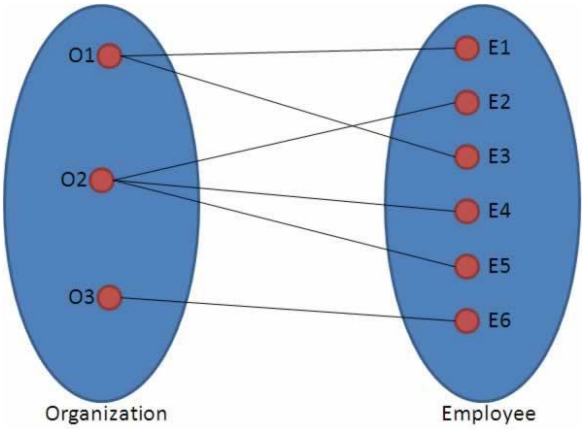


One employee is assigned with only one parking space and one parking space is assigned to only one employee. Hence it is a 1:1 relationship and cardinality is One-To-One (1:1)

In ER modeling, this can be mentioned using notations as given below

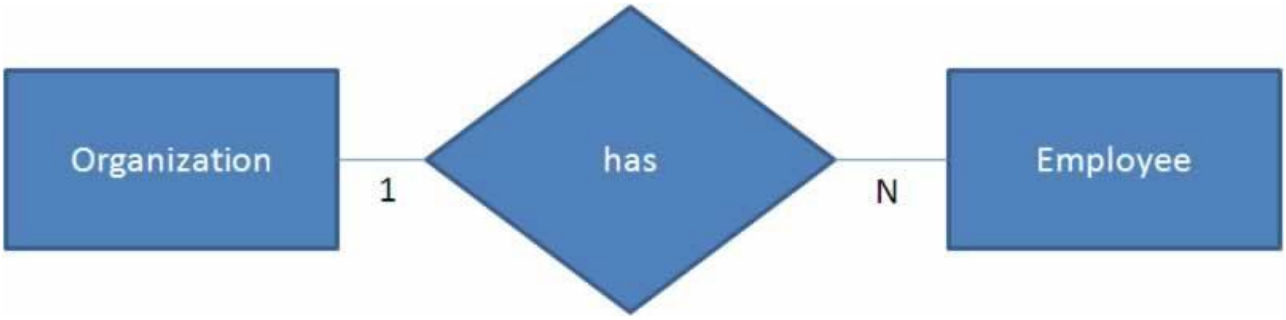
Example for Cardinality – One-to-Many (1:N)

Organization has employees



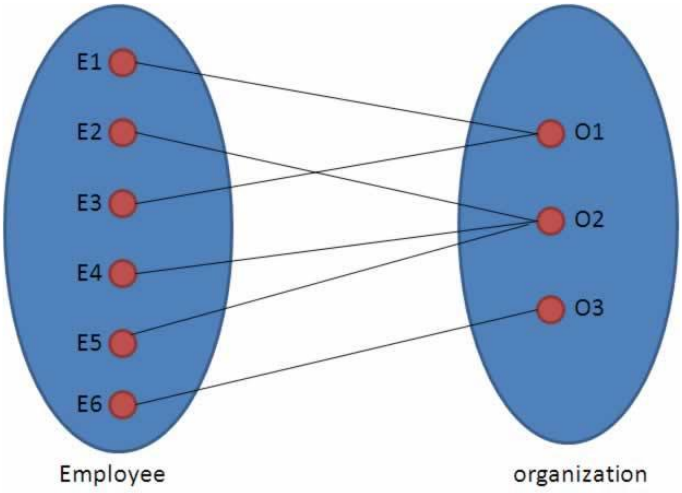
One organization can have many employees , but one employee works in only one organization. Hence it is a 1:N relationship and cardinality is One-To-Many (1:N)

In ER modeling, this can be mentioned using notations as given below



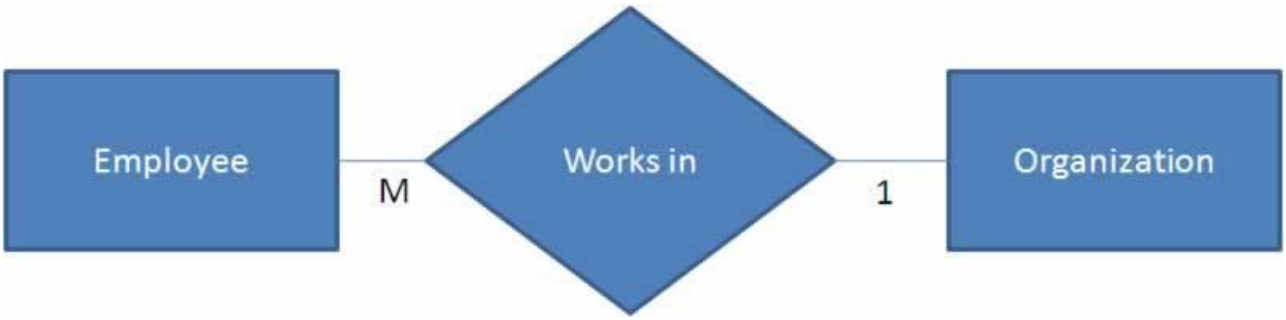
Example for Cardinality – Many-to-One (M :1)

It is the reverse of the One to Many relationship. employee works in organization



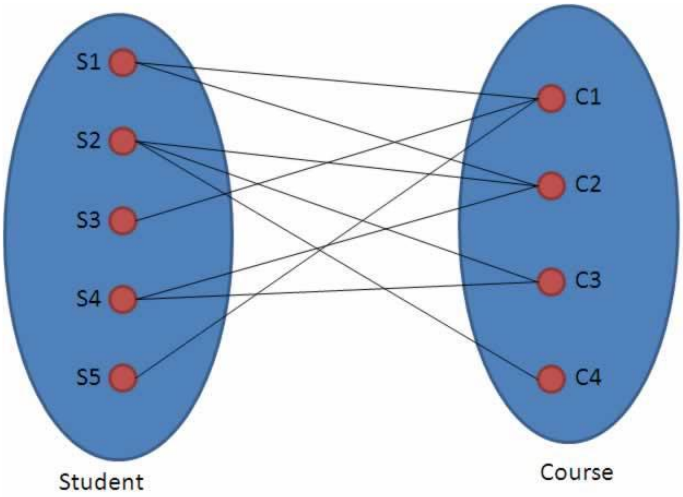
One employee works in only one organization But one organization can have many employees. Hence it is a M:1 relationship and cardinality is Many-to-One (M :1)

In ER modeling, this can be mentioned using notations as given below.



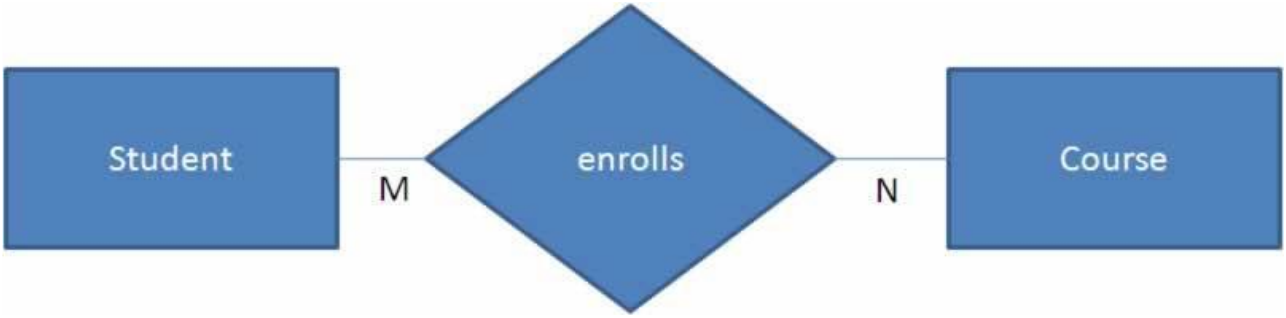
Example of Cardinality – Many-to-Many (M:N)

Students enrolls for courses



One student can enroll for many courses and one course can be enrolled by many students. Hence it is a M:N relationship and cardinality is Many-to-Many (M:N)

In ER modeling, this can be mentioned using notations as given below



Relationship Participation

1. Total

In total participation, every entity instance will be connected through the relationship to another instance of the other participating entity types

2. Partial

Example for relationship participation: Consider the relationship - Employee is head of the department.

Here all employees will not be the head of the department. Only one employee will be the head of the department. In other words, only few instances of employee entity participate in the above relationship. So employee entity's participation is partial in the said relationship.
However each department will be headed by some employee. So department entity's participation is total in the said relationship.

Advantages and Disadvantages of ER Modeling (Merits and Demerits of ER Modeling)

Advantages

1. ER Modeling is simple and easily understandable. It is represented in business users language and it can be understood by non-technical specialist.
2. Intuitive and helps in Physical Database creation.
3. Can be generalized and specialized based on needs.
4. Can help in database design.
5. Gives a higher level description of the system.

Disadvantages

1. Physical design derived from E-R Model may have some amount of ambiguities or inconsistency.
2. Sometime diagrams may lead to misinterpretations

Additional features of ER Model:

As the complexity of data increased, it became more and more difficult to use the traditional ER Model for database modelling. Hence some Additional Features were made to the existing ER Model to make it able to handle the complex applications better.

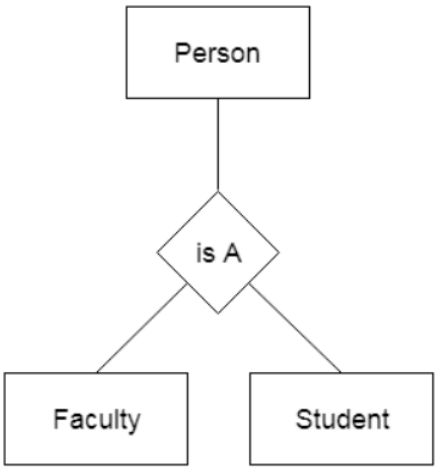
Hence, Three new concepts were added to the existing ER Model, they were:

- **Generalization**
- **Specialization**
- **Aggregration**

Generalization:

- Generalization is like a bottom-up approach in which two or more entities of lower level combine to form a higher level entity if they have some attributes in common.
- In generalization, an entity of a higher level can also combine with the entities of the lower level to form a further higher level entity.
- Generalization is more like subclass and superclass system, but the only difference is the approach. Generalization uses the bottom-up approach.
- In generalization, entities are combined to form a more generalized entity, i.e., subclasses are combined to make a superclass.

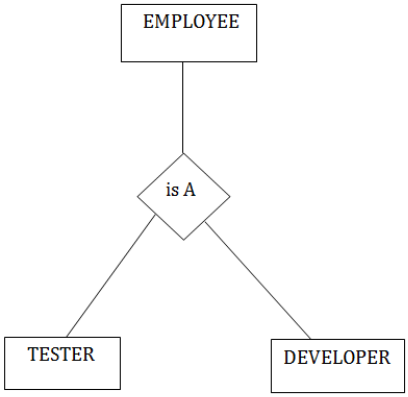
For example, Faculty and Student entities can be generalized and create a higher level entity Person.



Specialization

- Specialization is a top-down approach, and it is opposite to Generalization. In specialization, one higher level entity can be broken down into two lower level entities.
- Specialization is used to identify the subset of an entity set that shares some distinguishing characteristics.
- Normally, the superclass is defined first, the subclass and its related attributes are defined next, and relationship set are then added.

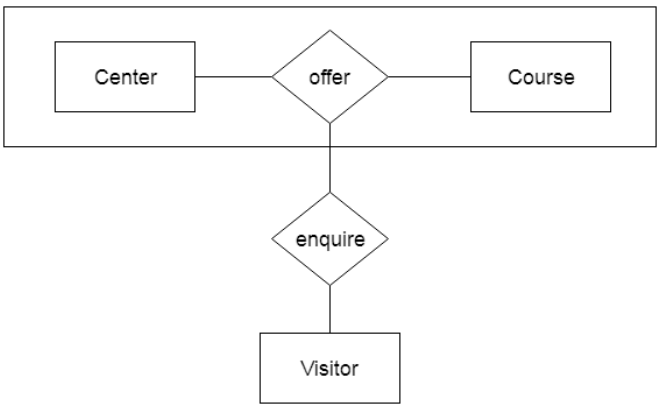
For example: In an Employee management system, EMPLOYEE entity can be specialized as TESTER or DEVELOPER based on what role they play in the company.



Aggregation:

In aggregation, the relation between two entities is treated as a single entity. In aggregation, relationship with its corresponding entities is aggregated into a higher level entity.

For example: Center entity offers the Course entity act as a single entity in the relationship which is in a relationship with another entity visitor. In the real world, if a visitor visits a coaching center then he will never enquiry about the Course only or just about the Center instead he will ask the enquiry about both.

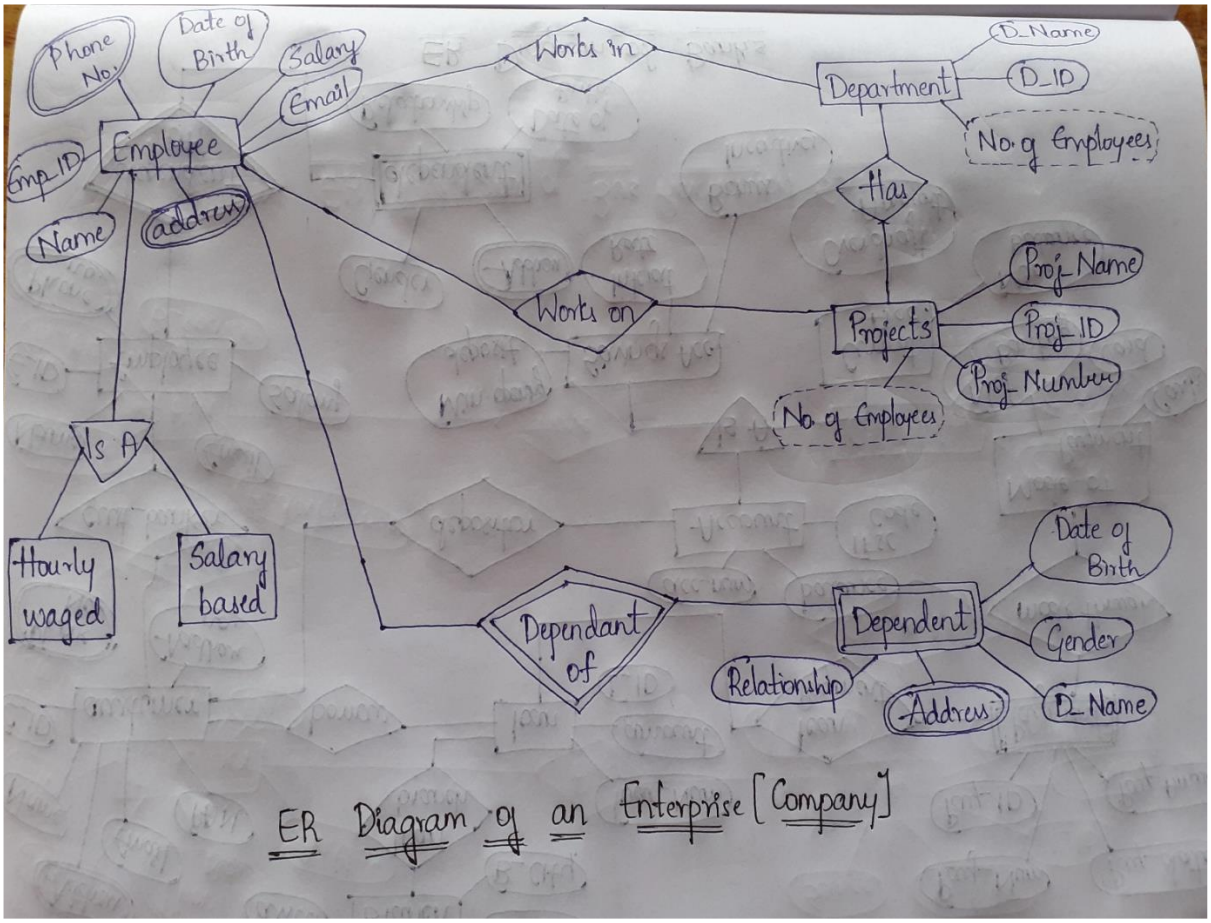


Conceptual Design for Large enterprises:

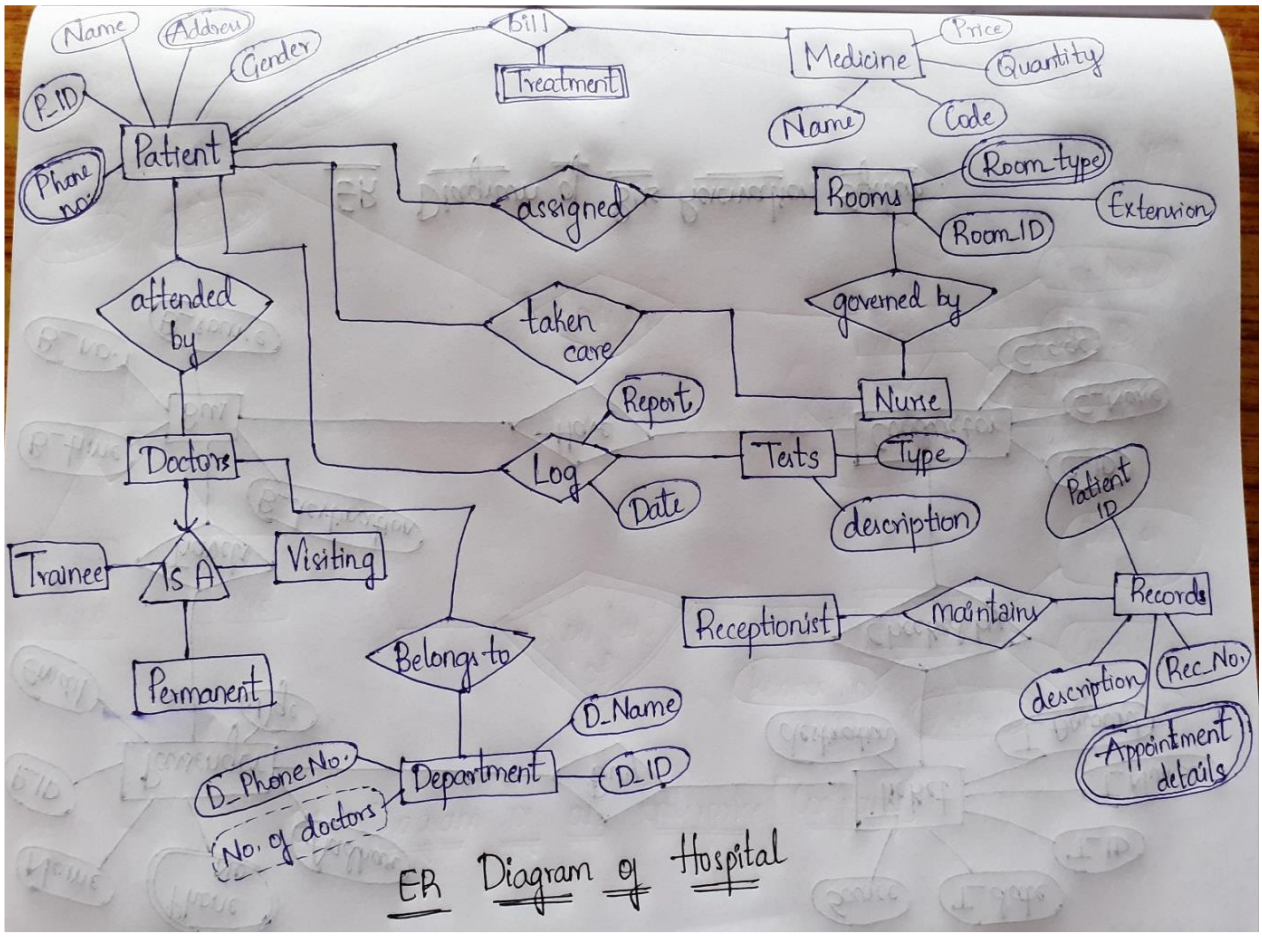
The process of conceptual design consists of more than just describing small fragments of the application in terms of ER diagrams. For a large enterprise, the design may require the efforts of more than one designer and span data and application code used by a number of user groups. Using a high-level, semantic data model such as ER diagrams for conceptual design in such an environment offers the additional advantage that the high-level design can be diagrammatically represented and is easily understood by the many people who must provide input to the design process.

An important aspect of the design process is the methodology used to structure the development of the overall design and to ensure that the design takes into account all user requirements and is consistent. The usual approach is that the requirements of various user groups are considered, any conflicting requirements are somehow resolved, and a single set of global requirements is generated at the end of the requirements analysis phase. Generating a single set of global requirements is a difficult task, but it allows the conceptual design phase to proceed with the development of a logical schema that spans all the data and applications throughout the enterprise.

An alternative approach is to develop separate conceptual schemas for different user groups and to then *integrate* these conceptual schemas. To integrate multiple conceptual schemas, we must establish correspondences between entities, relationships, and attributes, and we must resolve numerous kinds of conflicts (e.g., naming conflicts, domain mismatches, differences in measurement units). This task is difficult in its own right. In some situations, schema integration cannot be avoided. for example, when one organization merges with another, existing databases may have to be integrated. Schema integration is also increasing in importance as users demand access to *heterogeneous* data sources, often maintained by different organizations.



ER Diagram of an Enterprise [Company]



ER Diagram of Hospital